

Журналирование и анализ событий с помощью journalctl

Цель работы: Изучение возможностей утилиты *journalctl* для работы с системным журналом в Linux, поиск и анализ событий.

1. Просмотр и фильтрация событий

JournalD представляет собой системный демон журналов *SystemD*. *SystemD* спроектирован так, чтобы централизованно управлять системными логами от процессов, приложений и т.д. Все такие события обрабатываются демоном *JournalD*, который собирает логи со всей системы и сохраняет их в бинарных файлах.

Логирование событий в бинарном формате предоставляет ряд преимуществ. В частности, системные логи могут быть конвертированы в различные форматы. Кроме того, существует возможность детального отслеживания логов вплоть до отдельных событий с использованием фильтров по дате и времени.

Файлы логов *JournalD* могут собирать тысячи событий. При длительной работе системы объем таких логов может достигать нескольких гигабайт и более, поэтому анализ таких логов может происходить с задержками. В таком случае рекомендуется фильтровать вывод, чтобы ускорить обработку данных.

В *JournalD* есть утилита *journalctl*, которая позволяет искать и фильтровать записи по различным критериям.

Для того чтобы вывести все доступные журналы, достаточно ввести команду без дополнительных параметров:

```
# journalctl
```

По умолчанию журналы открываются в программе постраничного просмотра текста *less*. Чтобы выйти из программы *less*, нажмем клавишу *q*.

С помощью опции *-b* можно просмотреть все логи, собранные с момента последней загрузки системы:

```
# journalctl -b
```

Теперь выведем список предыдущих загрузок системы:

```
# journalctl --list-boots
```

В первой колонке указывается порядковый номер загрузки, во второй — ее идентификатор, в третьей — дата и время. Чтобы просмотреть лог для конкретной загрузки, можно использовать идентификаторы как из первой, так и из второй колонки.

Просмотрим логи с предпоследней загрузки системы:

```
# journalctl -b 1
```

По умолчанию вывод журналов будет отображать время событий согласно текущему часовому поясу, который настроен в системе. Однако, на самом деле, события в журналах сохраняются в универсальном координированном времени (UTC). Система будет сохранять события с метками времени в формате UTC, чтобы не возникало путаницы с переходами на летнее/зимнее время или изменениями часовых поясов.

Выведем журналы с временем в формате UTC:

```
# journalctl --utc
```

Для фильтрации по дате и времени важны следующие опции:

Опция `-S` (`--since`) указывает дату и время, начиная с которых необходимо отображать записи журнала.

Опция `-U` (`--until`) указывает дату и время, до которых необходимо отображать записи журнала.

В качестве значений для этих ключей можно использовать:

- формат `YYYY-MM-DD HH:MM:SS`;
- слова `"yesterday"`, `"today"`, `"tomorrow"`, `"now"`;
- удобочитаемые выражения вида `"10 hours ago"`, `"1 minute ago"`, `"50 minute ago"` и т.д.

Просмотрим логи за последние 20 минут:

```
# journalctl -S "20 minute ago"
```

Просмотрим логи с 10:00 до прошлого часа:

```
# journalctl -S "10:00" -U "1 hour ago"
```

Создадим логи, запустив и добавив в автозагрузку службу `sshd`:

```
# systemctl enable --now sshd
```

Теперь "случайно" допустим ошибку в конфигурационном файле `/etc/openssh/sshd_config`:

```
# vim /etc/openssh/sshd_config
```

```
# $OpenBSD: sshd_config,v 1.103 2018/04/09 20:41:22 tj Exp $
# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.
# This sshd was compiled with PATH=/bin:/usr/bin:/usr/local/bin
# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.
ort 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::
#HostKey /etc/openssh/ssh_host_rsa_key
#HostKey /etc/openssh/ssh_host_ecdsa_key
#HostKey /etc/openssh/ssh_host_ed25519_key
```

Сохраним изменения в файле.

Перезапустим службу:

```
# systemctl restart sshd
```

```
[root@alt ~]# systemctl restart sshd
Job for sshd.service failed because the control process exited with error code.
See "systemctl status sshd.service" and "journalctl -xeu sshd.service" for details.
```

В результате было получено сообщение об ошибке при попытке перезапустить службу *sshd*.

Чтобы диагностировать проблему, следует выполнить несколько шагов:

1. Проверим статус службы *sshd*:

```
# systemctl status sshd
```

```
[root@alt ~]# systemctl status sshd
* sshd.service - OpenSSH server daemon
   Loaded: loaded (/lib/systemd/system/sshd.service; enabled; vendor preset: enabled)
   Active: failed (Result: exit-code) since 11min ago
   Process: 4170 ExecStartPre=/usr/bin/ssh-keygen -A (code=exited, status=0/SUCCESS)
   Process: 4171 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=255/EXCEPTION)
   CPU: 5ms

alt systemd[1]: sshd.service: Scheduled restart job, restart counter is at 5.
alt systemd[1]: Stopped OpenSSH server daemon.
alt systemd[1]: sshd.service: Start request repeated too quickly.
alt systemd[1]: sshd.service: Failed with result 'exit-code'.
alt systemd[1]: Failed to start OpenSSH server daemon.
```

Команда покажет текущее состояние службы, последние логи и послужит подсказкой для диагностики.

2. Для получения дополнительной информации об ошибке, выполним следующую команду:

```
# journalctl -xeu sshd.service
```

Опция *-u* (*--unit*) позволяет фильтровать вывод, показывая только те логи, которые относятся к определенной системной службе.

Опция *-x* включает более подробные логи, в которых могут быть указаны дополнительные пояснения.

Опция -e (--pager-end) перемещает вывод к концу журнала.

Среди всех логов найдем следующие:

```
Начат процесс запуска юнита sshd.service.  
alt sshd[4171]: sshd: /etc/openssh/sshd_config: line 13: Bad configuration option: ort  
alt sshd[4171]: sshd: /etc/openssh/sshd_config: terminating, 1 bad configuration options  
alt systemd[1]: sshd.service: Control process exited, code=exited, status=255/EXCEPTION
```

Логи указывают на проблему в конфигурационном файле /etc/openssh/sshd_config. В 13-й строке конфигурации есть некорректный параметр или опции "ort".

Исправим ошибку и повторно перезапустим службу *sshd*:

```
# systemctl restart sshd
```

Для просмотра логов определенного процесса используется фильтр *_PID*. Просмотрим логи процесса *systemd* с *PID=1*:

```
# journalctl _PID=1
```

Для просмотра логов процессов, запущенных от имени определённого пользователя или группы, используются фильтры *_UID* и *_GID* соответственно. Предположим, у нас есть веб-сервер, запущенный от имени пользователя *user*. Сначала определим идентификатор пользователя *user*:

```
# id user
```

```
[root@alt ~]# id user  
uid=500(user) gid=500(user) группы=500(user),10(wheel),100(users),36(vboxusers),80(cdwriter),22(cdrom),81(audio),479(video),19(proc),  
83(radio),468(camera),71(floppy),498(xgrp),499(scanner),14(uucp),476(vboxusers),471(fuse),490(vboxadd),489(vboxsf)
```

Пользователь *user* имеет *UID=500*.

Теперь посмотрим логи процессов, запущенных от имени пользователя *user*:

```
# journalctl _UID=500
```

Для вывода всех идентификаторов групп, для которых есть записи в журналах, используется опция -F (--field):

```
# journalctl -F _GID
```

```
[root@alt ~]# journalctl -F _GID  
463  
471  
487  
488  
489  
474  
0  
500
```

Для просмотра логов ядра используется опция -k (--dmesg):

```
# journalctl -k
```

Приведенная команда покажет все логи ядра для текущей загрузки. Чтобы просмотреть логи ядра для предыдущих сессий, нужно воспользоваться опцией -b.

Посмотрим логи ядра предпоследней сессии загрузки системы:

```
# journalctl -k -b -1
```

Также можно фильтровать логи, указывая путь к файлу. Например, выведем все записи, относящиеся к исполняемому файлу `/bin/bash`:

```
# journalctl /bin/bash
```

Во время диагностики и исправления неполадок в системе нередко требуется просмотреть логи и выяснить, есть ли в них сообщения о критических ошибках.

В *JournalD* используется следующая классификация уровней приоритета:

Уровень	Обозначение	Номер	Описание
emergency	emerg	0	Критическая ошибка, система может быть не работоспособна.
alerts	alert	1	Ошибка, требующая немедленного вмешательства.
critical	crit	2	Критическая ошибка
errors	err	3	Ошибка, но не критическая
warning	warning	4	Предупреждение
notice	notice	5	Сообщение о нормальном, но значительном событии
info	info	6	Информационное сообщение
debug	debug	7	Отладочное сообщение

С помощью опции -p (--priority) можно фильтровать логи по их уровню приоритета.

Посмотрим все логи уровня приоритета *error*:

```
# journalctl -p err
```

Среди всех логов можно найти ошибки запуска сервиса *sshd*, рассмотренная нами выше:

```
alt systemd[1]: Failed to start OpenSSH server daemon.  
alt systemd[1]: Failed to start OpenSSH server daemon.
```

В *JournalD* формат вывода по умолчанию может варьироваться в зависимости от конфигурации системы, но обычно он выводит журналы в виде структурированного текста, который включает дату, время, имя службы, уровень сообщения и само сообщение.

С помощью опции `-o` (`--output`) можно преобразовывать данные логов в различные форматы, что облегчает их парсинг и дальнейшую обработку.

Представим данные логов сервиса *sshd* в формате *json*:

```
# journalctl -u sshd.service -o json
```

В результате логи в таком формате сложно анализировать.

Попробуем представить данные логов сервиса *sshd* в более человекочитаемом формате *json-pretty*:

```
# journalctl -u sshd.service -o json-pretty
```

Помимо *json* и *json-pretty* данные логов могут быть преобразованы в следующие форматы:

Формат *cat* выводит только сообщения из логов, без служебных полей и дополнительных данных. Это упрощенный формат, который полезен, если нужно видеть только текст сообщений.

Формат *export* используется для вывода журнала в бинарном формате, подходящем для резервного копирования или переноса данных между системами. Обычно этот формат не предназначен для чтения человеком и используется с командой *journalctl --import* для восстановления журнала.

Запишем в файл *backup.journal* все логи журналов в формате *export*:

```
# journalctl -o export > backup.journal
```

Формат *short* похожий на стандартный вывод *syslog*. В этом формате выводится самая необходимая информация: временные метки, служба, уровень важности и сообщение.

Формат *verbose* выводит максимально подробные данные о каждой записи журнала. Полезен для глубокого анализа и диагностики.

По умолчанию утилита *journalctl* показывает полные записи в постраничном режиме, и если строка длинная, она уходит вправо за пределы экрана. Чтобы увидеть скрытую часть, приходится использовать клавишу `→`. Чтобы длинные строки усекались и вместо пропущенной информации было троеточие, можно использовать опцию `--no-full`:

```
# journalctl --no-full
```

Чтобы вывести логи в стандартный вывод, воспользуемся опцией `--no-pager`:

```
# journalctl --no-pager
```

Следить за появлением новых сообщений можно с помощью опции `-f` (`--follow`):

```
# journalctl -f
```

Откроем веб-браузер Firefox. В результате в журнале увидим следующие сообщения:

```
alt dbus-daemon[2400]: [system] Activating via systemd: service name='org.freedesktop.timedate1' unit='dbus-org.freedesktop.timedate1.service' requested by ':1.112' (uid=500 pid=3570 comm="firefox --name firefox --class firefox")
alt systemd[1]: Starting Time & Date Service...
alt dbus-daemon[2400]: [system] Successfully activated service 'org.freedesktop.timedate1'
alt systemd[1]: Started Time & Date Service.
alt systemd[1]: systemd-timedated.service: Deactivated successfully.
```

В первой строке происходит активация сервиса *org.freedesktop.timedate1*. Данный сервис отвечает за работу в времени и датой в системе. В данном случае его активирует процесс *firefox* для запроса доступа к данным времени для последующей синхронизации. Во второй строке видно, что система начинает запускать сервис времени и даты. В третьей и четвертой строках подтверждается успешная активация и запуск этого сервиса. Пятая строка показывает, что через несколько секунд сервис *systemd-timedated*, управляющий временем в системе, был деактивирован, так как его работа завершена.

Чтобы вывести определенное количество последних записей, можно использовать опцию `-n`:

```
# journalctl -n
```

По умолчанию будут показаны последние 10 записей журнала.

Если необходимо вывести другое количество записей, укажите число после опции `-n`.

Выведем последние 15 записей журнала:

```
# journalctl -n 15
```

ЗАДАНИЕ

1. Просмотрите логи с приоритетом *warning* за последние 2 часа.
2. Выведите логи службы *sshd* упрощенном формате.
3. Просмотрите логи ядра за последние 3 загрузки системы.

2. Управление журналами

Все настройки *JournalD* находятся в файле */etc/systemd/journald.conf*:

```
# cat /etc/systemd/journald.conf
```

По умолчанию все директивы в файле */etc/systemd/journald.conf* закомментированы, т.е. все значения используются по умолчанию. Значение директивы, установленное в файле */etc/systemd/journald.conf* может быть переопределено в одном из файлов в директории */etc/systemd/journald.conf.d*.

Директива *Storage* задает метод хранения логов. Логи могут храниться в оперативной памяти в каталоге */var/log/journal* или в файловой системе на диске в каталоге */run/log/journal*. При хранении логов в оперативной памяти возможен просмотр логов только с момента последней загрузки системы.

Значения директивы *Storage*:

Параметр	Описание
<i>persistent</i>	Журналы будут храниться на диске в виде файлов.
<i>auto</i>	<i>JournalD</i> автоматически выберет метод хранения в зависимости от доступного пространства и настроек системы. Если поддерживается хранение на диске, журналы будут сохраняться в <i>/var/log/journal</i> . Если такая возможность отсутствует, то журналы будут храниться в <i>/run/log/journal</i> .
<i>volatile</i>	Журналы будут храниться только в памяти.
<i>none</i>	Журналы не будут сохраняться ни в каком виде.

Директива *Compress* включает или отключает сжатие сообщений перед записью в журнал. Принимает значения *yes* или *no*.

Директива *Seal* включает или отключает *Forward Secure Sealing* (FSS), которая позволяет накладывать криптографические отпечатки на журнал системных логов. Принимает значения *yes* или *no*.

Директива *SplitMode* определяет доступ к журналу пользователям.

Значения директивы *SplitMode*:

Параметр	Описание
<i>uid</i>	Все пользователи имеют доступ к журналу.
<i>login</i>	Пользователи имеют доступ только в сообщениям своего сеанса.
<i>none</i>	У пользователей нет доступа к журналу.

Директива *SyncIntervalSec* устанавливает таймаут, после которого происходит синхронизация и запись журнала на диск. Относится только к уровням *err*, *warning*, *notice*, *info*, *debug*. Сообщения уровня *emerg*, *alert*, *crit* сразу записываются на диск.

Директивы *RateLimitInterval* и *RateLimitBurst* настраивают ограничение скорости генерации сообщений для каждой службы. Если в интервале времени *RateLimitInterval* получено больше сообщений, чем указано в *RateLimitBurst*, все дальнейшие сообщения отбрасываются до окончания интервала. При этом генерируется сообщение о количестве отброшенных сообщений. Чтобы отключить ограничение скорости, нужно установить оба значения в ноль.

Директивы ротации логов управляют файлами логов. Основная цель ротации логов — предотвратить переполнение дискового пространства.

Единицы измерения: К (Килобайт), М (Мегабайт), Г (Гигабайт), Т (Терабайт), П (Петабайт), Е (Эксабайт).

Директива *SystemMaxUse* задает максимальный объем, который логи могут занимать в файловой системе на диске.

Директива *SystemKeepFree* определяет объем свободного места, которое должно оставаться в файловой системе на диске.

Директива *SystemMaxFileSize* указывает объем файла лога, по достижении которого он должен быть удален с диска.

Директива *SystemMaxFiles* определяет максимальное количество файлов журнала, которые могут храниться на диске.

Директива *RuntimeMaxUse* задает максимальный объем, который логи могут занимать в памяти.

Директива *RuntimeKeepFree* определяет объем свободного места, которое должно оставаться в памяти.

Директива *RuntimeMaxFileSize* указывает объем файла лога, по достижении которого он должен быть удален из памяти.

Директива *RuntimeMaxFiles* определяет максимальное количество файлов журнала, которые могут храниться в памяти.

Директива *MaxRetentionSec* определяет максимальное время хранения записей журнала. В директиве можно указывать следующие единицы измерения: *year*, *month*, *week*, *day*, *h* или *m*.

Директива *MaxFileSec* задает максимальное время хранения записей в одном файле журнала, после которого он переводится в следующий.

Директива *ForwardToSyslog* определяет, будет ли *JournalD* перенаправлять сообщения журналов в системный лог *Syslog*. Когда значение установлено на *no*, это означает, что журналы не будут отправляться в *Syslog*. Если установить на *yes*, сообщения будут пересылаться в систему *Syslog*.

Директива *ForwardToKMsg* указывает, будет ли *JournalD* отправлять сообщения в буфер сообщений ядра. Когда значение установлено на *no*, это

значит, что сообщения не будут отправляться в ядро. Установка на `yes` обеспечит отправку логов в ядро.

Директива *ForwardToConsole* определяет, будут ли сообщения журналов отображаться в консоли для текущих пользователей. Установив значение на `no`, вы отключаете вывод журналов в консоль. Установка на `yes` позволит пользователям видеть логи прямо в терминале.

Директива *ForwardToWall* указывает, должны ли сообщения журналов отправляться всем пользователям, подключенным к терминалу.

Директива *TTYPath* назначает консоль *TTY* для вывода сообщений, если установлена директива значение `yes` для директивы *ForwardToConsole*. По умолчанию используется `/dev/console`. Чтобы вывести на 12-ю консоль, необходимо установить `/dev/tty12`.

Директивы *MaxLevelStore*, *MaxLevelSyslog*, *MaxLevelKMsg*, *MaxLevelConsole*, *MaxLevelWall* определяют максимальный уровень сообщений, который сохраняется в журнал, выводится в журнал *Syslog*, буфер журнала ядра, консоль или стену.

Чтобы применить изменения в конфигурационном файле `/etc/systemd/journal.conf`, необходимо перезапустить службу *systemd-journald*:

```
# systemctl restart systemd-journald
```

Если журнал сохраняется при перезагрузке, его размер по умолчанию ограничен значением в 10% от объема соответствующей файловой системы и максимально может достигать 4 Гб. Например, для каталога `/var/log/journal`, расположенном на корневом разделе в 20 Гб, максимальный размер журналируемых данных составит 2 Гб. На разделе же 50Гб журнал может занимать дисковое пространство до 4 Гб.

Узнаем объем имеющихся на текущий момент логов:

```
# journalctl --disk-usage
```

Установим предельно допустимый размер для хранимых на диске логов:

```
# journalctl --vacuum-size=1G
```

```
[root@alt ~]# journalctl --vacuum-size=1G
Vacuuming done, freed 0B of archived journals from /var/log/journal/ef422b9313e0b69336a8215f6717ad4b.
Vacuuming done, freed 0B of archived journals from /var/log/journal.
Vacuuming done, freed 0B of archived journals from /run/log/journal.
```

Система не обнаружила старых архивированных журналов, которые можно было бы удалить, чтобы уменьшить общий размер хранилища до 1 Гб.

Теперь установим срок хранения логов, по истечении которого они будут автоматически удалены:

```
# journalctl --vacuum-time=2weeks
```

```
root@kali:~# journalctl --vacuum-time=2weeks
Vacuuming done, freed 0B of archived journals from /var/log/journal.
Vacuuming done, freed 0B of archived journals from /run/log/journal.
Deleted archived journal /var/log/journal/ef422b9313e0b69336a8215f6717ad4b/system@0006251286cf38ae-5c24b8999287c15d.journal~ (8.0M).
Deleted archived journal /var/log/journal/ef422b9313e0b69336a8215f6717ad4b/system@00062512dcf49915-3656ea9344910373.journal~ (8.0M).
Deleted archived journal /var/log/journal/ef422b9313e0b69336a8215f6717ad4b/system@0006251371043910-1881d74a6ad5ad3a.journal~ (8.0M).
Deleted archived journal /var/log/journal/ef422b9313e0b69336a8215f6717ad4b/system@00062513f87ac8d0-7dc0cf0c711966c4.journal~ (8.0M).
Deleted archived journal /var/log/journal/ef422b9313e0b69336a8215f6717ad4b/system@00062518d3dfde8b-705d750d7459b421.journal~ (8.0M).
Deleted archived journal /var/log/journal/ef422b9313e0b69336a8215f6717ad4b/system@0006251930131b6d-3c91e6ad1d82d9e3.journal~ (8.0M).
Deleted archived journal /var/log/journal/ef422b9313e0b69336a8215f6717ad4b/system@0006270812f1bd2f-4414ab43f7f5f2ed.journal~ (8.0M).
Vacuuming done, freed 56.0M of archived journals from /var/log/journal/ef422b9313e0b69336a8215f6717ad4b.
```

Разберем вывод:

Строки “Deleted archived journal ... (8.0M)” указывают на то, что были удалены несколько архивных файлов журналов, каждый из которых имел размер 8.0 Мб. Данные файлы были записаны более двух недель назад и подлежали удалению в соответствии с запросом.

Строка “Vacuuming done, freed 56.0M of archived journals ...” указывает на то, что было освобождено 56.0 МБ пространства за счет удаления старых архивов журналов из заданного каталога.

Настройки ротации логов можно также прописать в конфигурационном файле `/etc/systemd/journald.conf`.

ЗАДАНИЕ

Настройте *JournalD* в соответствии с указанными требованиями:

1. Установите максимальный размер хранилища журналов *500 Мб*.
2. Ограничьте максимальный срок хранения журналов *10* днями.
3. Установите параметр таким образом, чтобы при достижении *75%* от выделенного пространства начиналась ротация журналов.
4. Настройте сохранение журналов при перезагрузке системы.
5. Настройте уровень ведения журналов на *info*.